

Connected to Python

```
In [ ]: import numpy as np
import pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import warnings
import pmdarima as pm # para halar el mejor modelo de ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox # para la prueba de auto
warnings.filterwarnings('ignore')
from statsmodels.tsa.stattools import adfuller # para el test de raiz unitaria
from statsmodels.stats.diagnostic import het_arch
from statsmodels.stats.stattools import jarque_bera
from scipy.stats import norm
import scipy.stats as stats
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
In [ ]: CARPETA_ACTUAL = Path('.').resolve()

# parent sube un nivel (sale de 'codigo') y luego entra a 'bases_de_datos'
ruta_hipotecas = CARPETA_ACTUAL.parent / "bases_de_datos" / "Hipotecas_MORTGAGE3

#Carga la base de datos
df_hipotecas = pd.read_csv(ruta_hipotecas, parse_dates=['observation_date'], ind
```

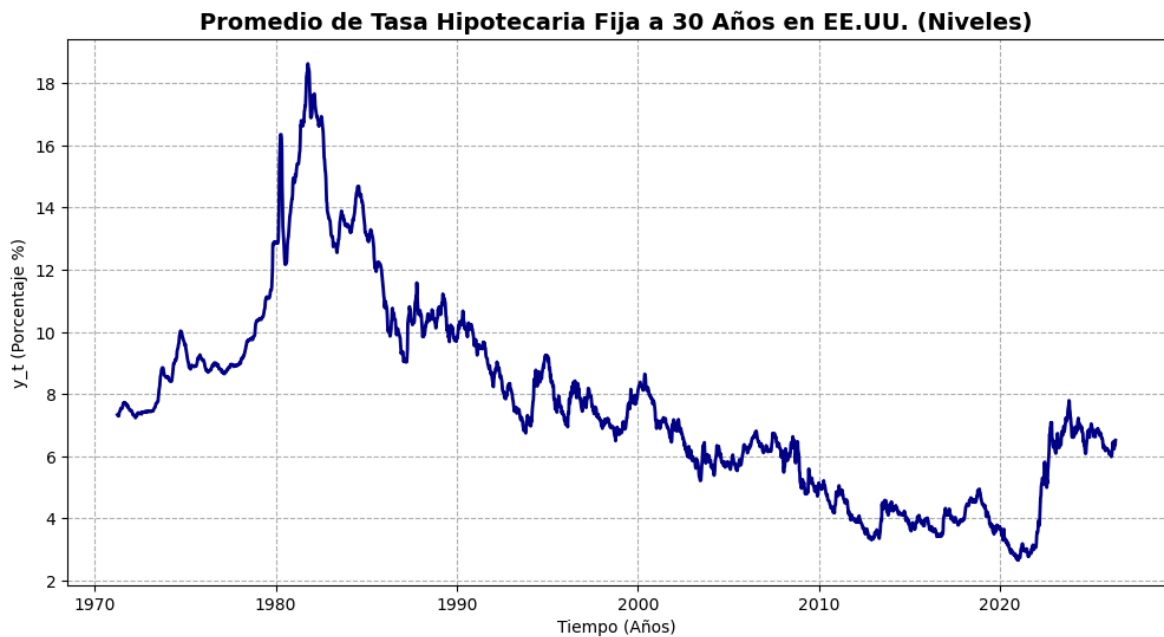
```
In [ ]: # Renombro las columnas por que tiene un nombre muy feo y tedioso
df_hipotecas = df_hipotecas.rename(columns={'MORTGAGE30US': 'Tasa_hipotecaria'})
print(df_hipotecas.info())
print(df_hipotecas.describe())
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 2878 entries, 1971-04-02 to 2026-05-21
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Tasa_hipotecaria  2878 non-null   float64
dtypes: float64(1)
memory usage: 45.0 KB
None
```

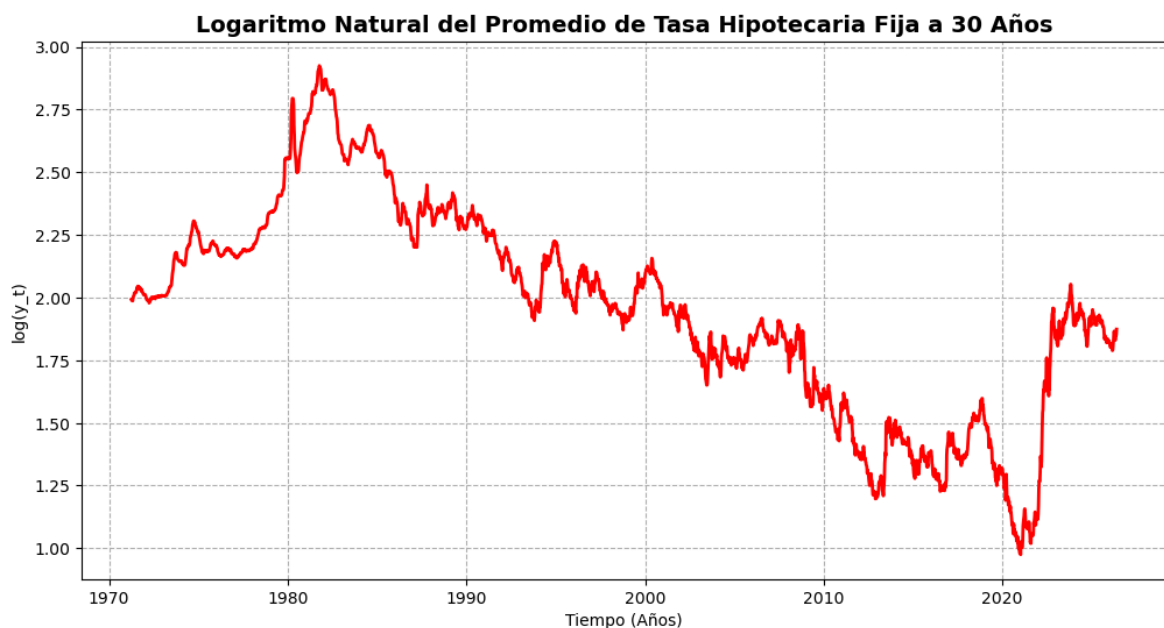
	Tasa_hipotecaria
count	2878.000000
mean	7.687224
std	3.187367
min	2.650000
25%	5.472500
50%	7.230000
75%	9.237500
max	18.630000

```
In [ ]: plt.figure(figsize=(12, 6))
plt.plot(df_hipotecas.index, df_hipotecas['Tasa_hipotecaria'], color='darkblue',
plt.title('Promedio de Tasa Hipotecaria Fija a 30 Años en EE.UU. (Niveles)', fon
plt.xlabel('Tiempo (Años)')
plt.ylabel('y_t (Porcentaje %)')
plt.grid(True, linestyle='--')
plt.show()
```

```
# Nota : Se puede apreciar que no tiene ni media ni varianza constante, por lo q
# Por ello voy a aplicar primer logaritmo y luego diferencia para intentar hace
#
```



```
In [ ]: df_hipotecas['log_hipotecas'] = np.log(df_hipotecas['Tasa_hipotecaria'])
# Se anade un columna de Log de Los datos en La base de datos
plt.figure(figsize=(12, 6))
plt.plot(df_hipotecas.index, df_hipotecas['log_hipotecas'], color='red', linewidth=2)
plt.title('Logaritmo Natural del Promedio de Tasa Hipotecaria Fija a 30 Años', fontweight='bold')
plt.xlabel('Tiempo (Años)')
plt.ylabel('log(y_t)')
plt.grid(True, linestyle='--')
plt.show()
```



```
In [ ]: plt.figure(figsize=(12, 6))

# 1. Grafica serie original (Rojo)
plt.plot(df_hipotecas.index, df_hipotecas['Tasa_hipotecaria'], color='red', linewidth=2)
plt.ylabel('Tasa Original (%)', color='red')
```

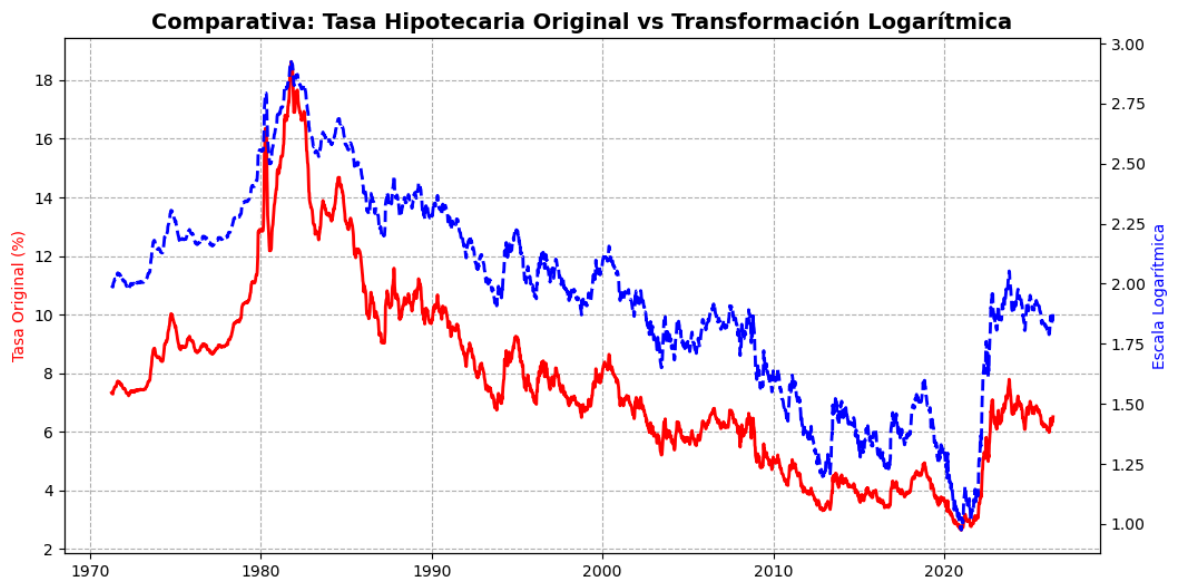
```
plt.grid(True, linestyle='--')

# 2. Activar el segundo eje Y compartiendo el mismo eje X
plt.twinx()

# 3. Grafica serie en logaritmo (Azul)
plt.plot(df_hipotecas.index, df_hipotecas['log_hipotecas'], color='blue', linewidth=2)
plt.ylabel('Escala Logarítmica', color='blue')

# Titulos
plt.title('Comparativa: Tasa Hipotecaria Original vs Transformación Logarítmica')
plt.xlabel('Tiempo (Años)')

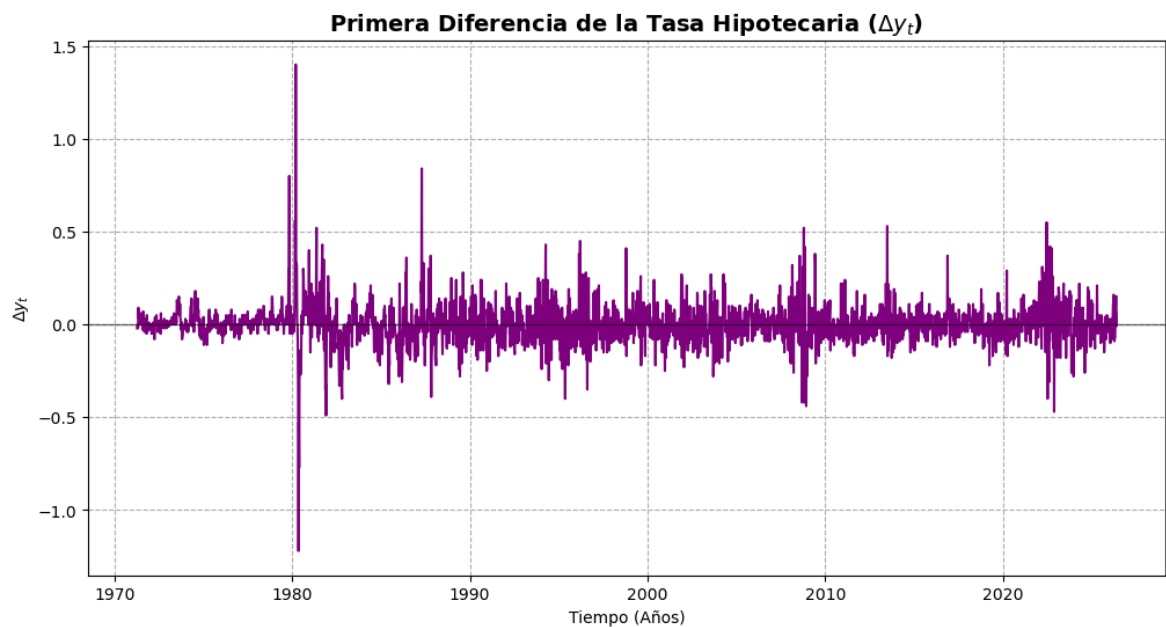
plt.show()
```



```
In [ ]: # Se añade la columna de diferencia a la base de datos
df_hipotecas['Dif_hipotecas'] = df_hipotecas['Tasa_hipotecaria'].diff()

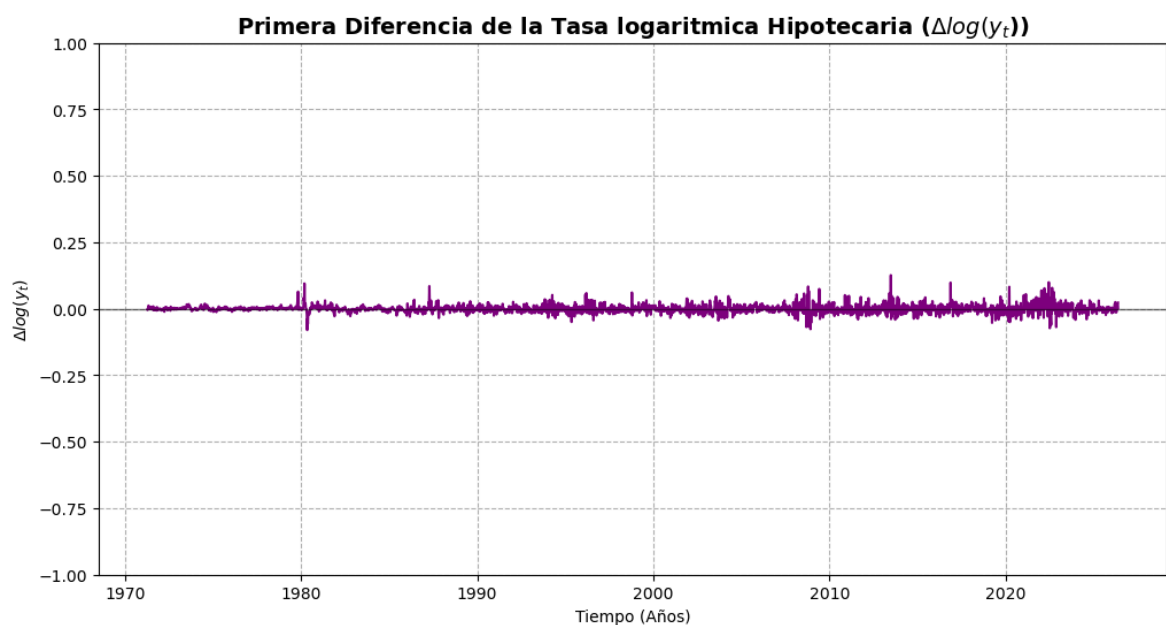
plt.figure(figsize=(12, 6))
plt.plot(df_hipotecas.index, df_hipotecas['Dif_hipotecas'], color='purple', line
plt.title('Primera Diferencia de la Tasa Hipotecaria ($\Delta y_t$)', fontsize=12)
plt.xlabel('Tiempo (Años)')
plt.ylabel('$_{\Delta y_t}$')
plt.axhline(0, color='black', linestyle='--', alpha=0.5, linewidth=1) # Línea de
plt.grid(True, linestyle='--')
plt.show()

# mejora :D entonces creo que por el momento es un ARIMA(?,1,?)
#recapitulando no es necesario aplicar logaritmo pero si aplicar diferencia
```



```
In [ ]: df_hipotecas['Dif_log_hipotecas'] = df_hipotecas['log_hipotecas'].diff()

plt.figure(figsize=(12, 6))
plt.plot(df_hipotecas.index, df_hipotecas['Dif_log_hipotecas'], color='purple',
plt.title('Primera Diferencia de la Tasa logaritmica Hipotecaria ( $\Delta \log(y_t)$ '))
plt.xlabel('Tiempo (Años)')
plt.ylabel('( $\Delta \log(y_t)$ )')
plt.ylim(-1, 1)
plt.axhline(0, color='black', linestyle='--', alpha=0.5, linewidth=1) # Línea de
plt.grid(True, linestyle='--')
plt.show()
```



```
In [ ]: # Toca quitar el primer dato por la diferencia
serie_diferenciada = df_hipotecas['Dif_log_hipotecas'].dropna()

plt.figure(figsize=(16, 5))

#Grafico FAC
plt.subplot(1, 2, 1)
plot_acf(serie_diferenciada, lags=40, zero=False, ax=plt.gca(), color='darkblue')
plt.title('Función de Autocorrelación (FAC)', fontsize=13, fontweight='bold')
```

```

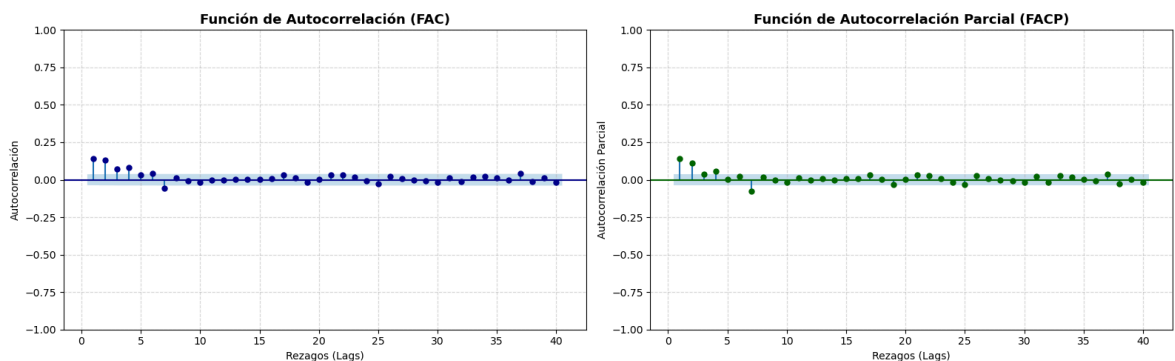
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación')
plt.grid(True, linestyle='--', alpha=0.5)

#Grafico FACP
plt.subplot(1, 2, 2)
plot_pacf(serie_diferenciada, lags=40, zero=False, ax=plt.gca(), color='darkgreen')
plt.title('Función de Autocorrelación Parcial (FACP)', fontsize=13, fontweight='bold')
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación Parcial')
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

# yo diria que puede ser un ARIMA(2,1,2) por parcimonia ya que tanto
# La parte autoregresiva como la de media movil parecieran que tiene solucion im

```



```

In [ ]: print("Buscando el mejor modelo ARIMA segun el criterio de AIC...")

# Ejecutamos el auto_arima
modelo_automatgico = pm.auto_arima(df_hipotecas['log_hipotecas'],
                                   start_p=0, start_q=0, # Desde qué rezagos empezamos
                                   max_p=5, max_q=5,      # Hasta qué rezago máximo
                                   d=1,                  # Ya comprobamos que necesitamos
                                   seasonal=False,        # No es un modelo estacional
                                   trace=True,            # Clave: Muestra la "comparación de modelos"
                                   error_action='ignore',
                                   suppress_warnings=True,
                                   stepwise=True)         # Algoritmo eficiente de búsqueda

print("\n--- RESUMEN DEL MODELO GANADOR ---")
print(modelo_automatgico.summary())
# SOy muy bueno le atine a la primera :D ARIMA(2,1,2) es el mejor modelo segun el criterio de AIC
# traiganme ya el otro parcial :) y lo cinquo jaja

```

Buscando el mejor modelo ARIMA segun el criterio de AIC...

Performing stepwise search to minimize aic

```
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=-15887.619, Time=0.39 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=-15942.287, Time=0.29 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AIC=-15931.558, Time=0.79 sec
ARIMA(0,1,0)(0,0,0)[0] : AIC=-15889.598, Time=0.18 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AIC=-15976.372, Time=0.98 sec
ARIMA(3,1,0)(0,0,0)[0] intercept : AIC=-15978.790, Time=1.04 sec
ARIMA(4,1,0)(0,0,0)[0] intercept : AIC=-15986.267, Time=1.11 sec
ARIMA(5,1,0)(0,0,0)[0] intercept : AIC=-15984.289, Time=1.68 sec
ARIMA(4,1,1)(0,0,0)[0] intercept : AIC=-15984.211, Time=1.87 sec
ARIMA(3,1,1)(0,0,0)[0] intercept : AIC=-15976.128, Time=2.21 sec
ARIMA(5,1,1)(0,0,0)[0] intercept : AIC=-15982.258, Time=1.14 sec
ARIMA(4,1,0)(0,0,0)[0] : AIC=-15988.259, Time=1.32 sec
ARIMA(3,1,0)(0,0,0)[0] : AIC=-15980.780, Time=0.36 sec
ARIMA(5,1,0)(0,0,0)[0] : AIC=-15986.281, Time=0.79 sec
ARIMA(4,1,1)(0,0,0)[0] : AIC=-15986.203, Time=0.65 sec
ARIMA(3,1,1)(0,0,0)[0] : AIC=-15978.117, Time=0.65 sec
ARIMA(5,1,1)(0,0,0)[0] : AIC=-15984.249, Time=0.87 sec
```

Best model: ARIMA(4,1,0)(0,0,0)[0]

Total fit time: 16.339 seconds

--- RESUMEN DEL MODELO GANADOR ---

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:      2878
Model:                 SARIMAX(4, 1, 0)      Log Likelihood      7999.129
Date:                  Sun, 24 May 2026      AIC      -15988.259
Time:                  17:21:50      BIC      -15958.436
Sample:                0      HQIC      -15977.509
                        - 2878
```

Covariance Type: opg

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.1175      0.011     10.760      0.000      0.096      0.139
ar.L2          0.1007      0.011      8.793      0.000      0.078      0.123
ar.L3          0.0323      0.012      2.680      0.007      0.009      0.056
ar.L4          0.0574      0.012      4.682      0.000      0.033      0.081
sigma2         0.0002     2.87e-06     78.385      0.000      0.000      0.000
=====
```

```
==
Ljung-Box (L1) (Q):      0.00      Jarque-Bera (JB):      6174.
66
Prob(Q):      0.99      Prob(JB):      0.
00
Heteroskedasticity (H):      5.00      Skew:      0.
78
Prob(H) (two-sided):      0.00      Kurtosis:      10.
00
=====
==
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
In [ ]: print("Buscando el mejor modelo ARIMA basándose en el BIC...")
```

```
# Ejecutamos el auto_arima cambiando el criterio de selección al BIC
modelo_automatiko = pm.auto_arima(df_hipotecas['log_hipotecas'],
                                  start_p=0, start_q=0,
                                  max_p=5, max_q=5,
                                  d=1,
                                  seasonal=False,
                                  information_criterion='bic', # <-- CLAVE: Fuer
                                  trace=True,
                                  error_action='ignore',
                                  suppress_warnings=True,
                                  stepwise=True)

print("\n--- RESUMEN DEL MODELO GANADOR (SEGÚN BIC) ---")
print(modelo_automatiko.summary())
```

Buscando el mejor modelo ARIMA basándose en el BIC...

Performing stepwise search to minimize bic

```
ARIMA(0,1,0)(0,0,0)[0] intercept : BIC=-15875.690, Time=0.32 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : BIC=-15924.393, Time=0.31 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : BIC=-15913.665, Time=0.82 sec
ARIMA(0,1,0)(0,0,0)[0] : BIC=-15883.634, Time=0.24 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : BIC=-15952.514, Time=0.97 sec
ARIMA(3,1,0)(0,0,0)[0] intercept : BIC=-15948.968, Time=1.04 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : BIC=-15952.911, Time=1.00 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : BIC=-15958.719, Time=0.54 sec
ARIMA(1,1,2)(0,0,0)[0] intercept : BIC=-15953.549, Time=0.92 sec
ARIMA(0,1,2)(0,0,0)[0] intercept : BIC=-15941.473, Time=2.35 sec
ARIMA(2,1,2)(0,0,0)[0] intercept : BIC=-15950.354, Time=1.22 sec
ARIMA(1,1,1)(0,0,0)[0] : BIC=-15966.877, Time=0.95 sec
ARIMA(0,1,1)(0,0,0)[0] : BIC=-15921.613, Time=0.30 sec
ARIMA(1,1,0)(0,0,0)[0] : BIC=-15932.343, Time=0.15 sec
ARIMA(2,1,1)(0,0,0)[0] : BIC=-15960.868, Time=0.77 sec
ARIMA(1,1,2)(0,0,0)[0] : BIC=-15961.570, Time=0.59 sec
ARIMA(0,1,2)(0,0,0)[0] : BIC=-15949.424, Time=0.43 sec
ARIMA(2,1,0)(0,0,0)[0] : BIC=-15960.467, Time=0.30 sec
ARIMA(2,1,2)(0,0,0)[0] : BIC=-15958.310, Time=0.60 sec
```

Best model: ARIMA(1,1,1)(0,0,0)[0]

Total fit time: 13.828 seconds

--- RESUMEN DEL MODELO GANADOR (SEGÚN BIC) ---

SARIMAX Results

```
=====
Dep. Variable: y No. Observations: 2878
Model: SARIMAX(1, 1, 1) Log Likelihood 7995.385
Date: Sun, 24 May 2026 AIC -15984.771
Time: 17:23:23 BIC -15966.877
Sample: 0 HQIC -15978.321
- 2878
```

Covariance Type: opg

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.7505        0.035     21.333      0.000         0.682         0.819
ma.L1         -0.6242        0.040    -15.493      0.000        -0.703        -0.545
sigma2          0.0002      2.8e-06     80.568      0.000         0.000         0.000
=====
```

```
==
Ljung-Box (L1) (Q):          0.24  Jarque-Bera (JB):          6355.
34
Prob(Q):          0.63  Prob(JB):          0.
00
Heteroskedasticity (H):          5.00  Skew:          0.
77
Prob(H) (two-sided):          0.00  Kurtosis:          10.
12
=====
==
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
In [ ]: modelo_arima_111 = ARIMA(df_hipotecas['log_hipotecas'], order=(1, 1, 1))
resultados_111 = modelo_arima_111.fit()
```



```
print(resultados_111.summary())
```

```

SARIMAX Results
=====
Dep. Variable:          log_hipotecas    No. Observations:          2878
Model:                  ARIMA(1, 1, 1)    Log Likelihood              7995.385
Date:                   Sun, 24 May 2026  AIC                          -15984.771
Time:                   17:26:45          BIC                          -15966.877
Sample:                 0                HQIC                         -15978.321
                                - 2878
Covariance Type:        opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.7505      0.035      21.333      0.000      0.682      0.819
ma.L1         -0.6242      0.040     -15.493      0.000     -0.703     -0.545
sigma2          0.0002     2.8e-06     80.568      0.000      0.000      0.000
=====
==
Ljung-Box (L1) (Q):                0.24    Jarque-Bera (JB):                6355.
34
Prob(Q):                            0.63    Prob(JB):                            0.
00
Heteroskedasticity (H):              5.00    Skew:                                0.
77
Prob(H) (two-sided):                0.00    Kurtosis:                            10.
12
=====
==

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

```

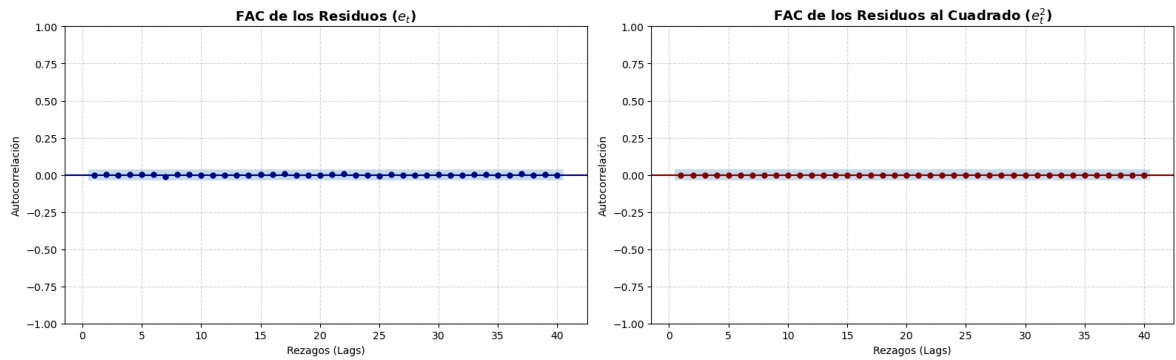
In [ ]: # se sacan los residuos del modelo :D
residuos = resultados_111.resid
residuos_cuadrado = residuos ** 2

# para que entren los dos graficos en la misma figura
plt.figure(figsize=(16, 5))

# FAC de los residuos
plt.subplot(1, 2, 1)
plot_acf(residuos, lags=40, zero=False, ax=plt.gca(), color='darkblue')
plt.title('FAC de los Residuos ($e_t$)', fontsize=13, fontweight='bold')
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación')
plt.grid(True, linestyle='--', alpha=0.5)

# fac de los reiduos aal cuadrado
plt.subplot(1, 2, 2)
plot_acf(residuos_cuadrado, lags=40, zero=False, ax=plt.gca(), color='darkred')
plt.title('FAC de los Residuos al Cuadrado ($e_t^2$)', fontsize=13, fontweight='bold')
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```



```
In [ ]: ljung_box = acorr_ljungbox(
        residuos,
        lags=[6, 12, 18, 24],
        return_df=True
    )

    print("Prueba Ljung-Box")
    print(ljung_box)

    if (ljung_box['lb_pvalue'] > 0.05).all():
        print("No se rechaza H0; no hay autocorrelación en los residuos.")
    else:
        print("Se rechaza H0; hay autocorrelación en los residuos.")
    # H0: no hay autocorrelación en los residuos.
    # Si p-value > 0.05, no se rechaza H0.
```

Prueba Ljung-Box

	lb_stat	lb_pvalue
6	0.190930	0.999865
12	0.626297	0.999999
18	0.803128	1.000000
24	0.928988	1.000000

No se rechaza H0; no hay autocorrelación en los residuos.

```
In [ ]: arch_test = het_arch(residuos, nlags=12)

    print("Prueba ARCH")
    print("LM statistic:", arch_test[0])
    print("LM p-value:", arch_test[1])

    if arch_test[1] > 0.05:
        print("No se rechaza H0; no hay efectos ARCH.")
    else:
        print("Se rechaza H0; hay efectos ARCH.")

    # H0: no hay efectos ARCH.
    # Si p-value > 0.05, no se rechaza H0.
```

Prueba ARCH

LM statistic: 316.96770839463153
 LM p-value: 1.2761799040765053e-60
 Se rechaza H0; hay efectos ARCH.

```
In [ ]: jb_stat, jb_pvalue, asimetria, curtosis = jarque_bera(residuos)

    print("Prueba Jarque-Bera")
    print("Jarque-Bera statistic:", jb_stat)
    print("Jarque-Bera p-value:", jb_pvalue)
```

```

if jb_pvalue > 0.05:
    print("No se rechaza H0; los residuos siguen una distribución normal.")
else:
    print("Se rechaza H0; los residuos no siguen una distribución normal.")
# H0: Los residuos siguen una distribución normal.
# Si p-value > 0.05, no se rechaza H0.

```

Prueba Jarque-Bera

Jarque-Bera statistic: 540243663.9713585

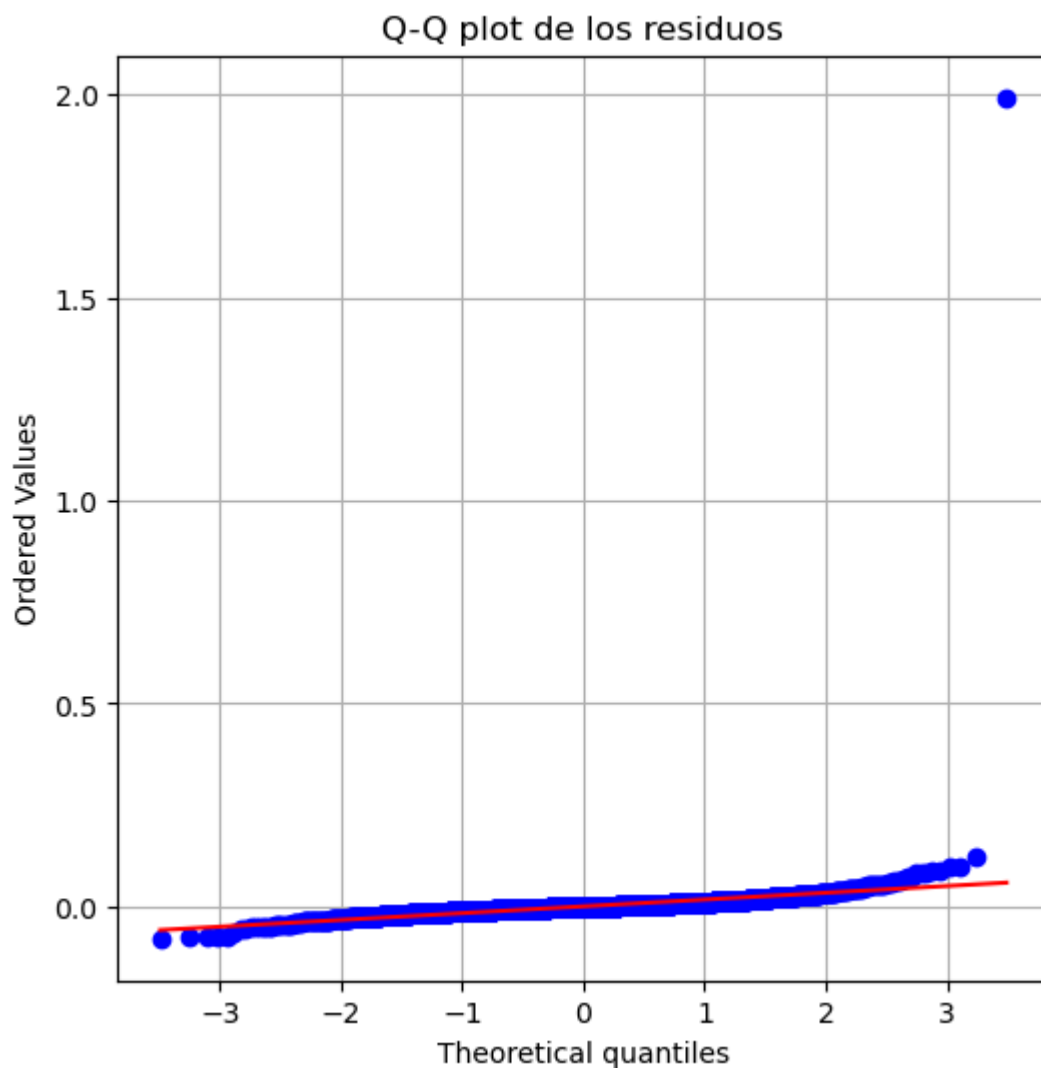
Jarque-Bera p-value: 0.0

Se rechaza H0; los residuos no siguen una distribución normal.

```

In [ ]: plt.figure(figsize=(6, 6))
stats.probplot(residuos, dist="norm", plot=plt)
plt.title("Q-Q plot de los residuos")
plt.grid(True)
plt.show()

```



```

In [ ]: # errores del modelo
residuos = resultados_111.resid

plt.figure(figsize=(12, 5))

# grafica de los residuos
plt.plot(df_hipotecas.index, residuos, color='darkwhite' if False else 'dimgray')

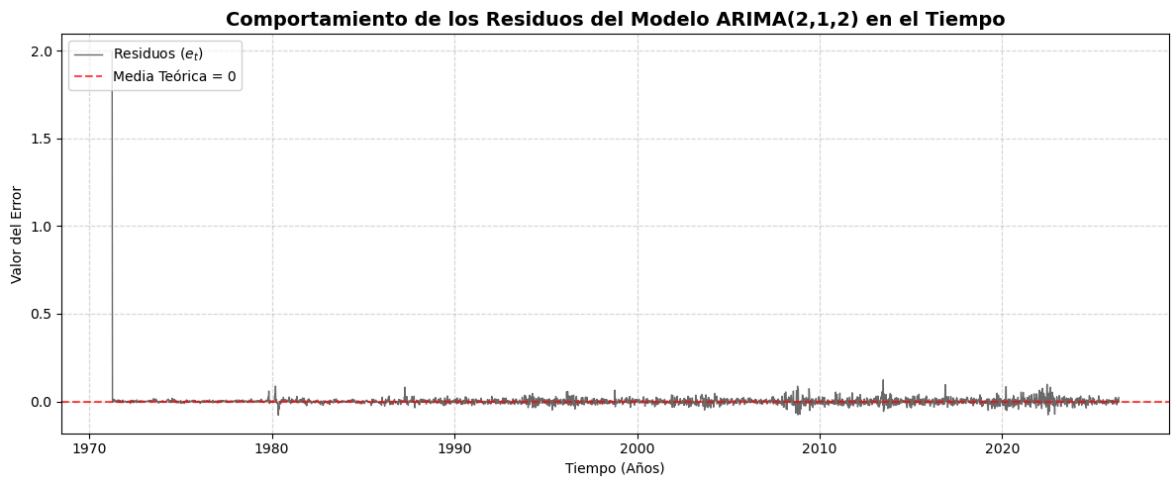
# linea de referencia en cero

```

```
plt.axhline(0, color='red', linestyle='--', alpha=0.7, linewidth=1.5, label='Med

# Configuración estética para el póster
plt.title('Comportamiento de los Residuos del Modelo ARIMA(2,1,2) en el Tiempo',
plt.xlabel('Tiempo (Años)')
plt.ylabel('Valor del Error')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(loc='upper left')

plt.tight_layout()
plt.show()
```



```
In [ ]: df_hipotecas = df_hipotecas.asfreq(df_hipotecas.index.inferred_freq)

# configurar del pronostico
modelo_final = SARIMAX(
    df_hipotecas['log_hipotecas'],
    order=(2, 1, 2),
    trend='c',
    enforce_stationarity=False,
    enforce_invertibility=False
)
resultado_final = modelo_final.fit(dis= False)

# pronosticar 10 periodos
pasos_futuros = 10
pronostico_objeto = resultado_final.get_forecast(steps=pasos_futuros)

# extraer la media y los intervalos
pronostico_nivel = pronostico_objeto.predicted_mean
intervalos_nivel = pronostico_objeto.conf_int()

# imprimir tabla
tabla_pronostico = pd.DataFrame({
    "Pronóstico (%)": pronostico_nivel,
    "Límite Inferior": intervalos_nivel.iloc[:, 0],
    "Límite Superior": intervalos_nivel.iloc[:, 1]
})
print("\n--- TABLA DE PRONÓSTICOS GENERADA CON ÉXITO ---")
print(tabla_pronostico.round(4))
```

--- TABLA DE PRONÓSTICOS GENERADA CON ÉXITO ---

	Pronóstico (%)	Límite Inferior	Límite Superior
2026-05-22	1.8744	1.8659	1.8829
2026-05-23	1.8753	1.8624	1.8881
2026-05-24	1.8756	1.8585	1.8927
2026-05-25	1.8760	1.8554	1.8966
2026-05-26	1.8761	1.8522	1.9000
2026-05-27	1.8762	1.8494	1.9031
2026-05-28	1.8763	1.8466	1.9059
2026-05-29	1.8764	1.8442	1.9085
2026-05-30	1.8764	1.8418	1.9109
2026-05-31	1.8764	1.8396	1.9131